

What Do Object Detectors Learn from Synthetic Data?

Architecture Baselines, Synthetic-to-Real Transfer, and Interpretability on the DIMO Dataset

Authors: Tim Barvenov, Janek Majchrzycki

Date: 12.06.2026

Repository: [GitHub](#)

Results, artifacts and model checkpoints: [Google Drive](#)

Abstract

This project investigates what object detectors learn when trained on real and synthetic data. We use DIMO, a synthetic-real industrial object dataset, and formulate the task as object detection under a synthetic-to-real transfer setting. Following the revised experimental design discussed with the instructor, we separate the study into two stages. First, we build an architecture baseline by fine-tuning 30 detector variants on real-only images and evaluating them on a shared real test set. Second, we select five representative detectors and retrain them across five data regimes: real-only, all synthetic, weak synthetic randomization, strong synthetic randomization, and a 50/50 real-synthetic mixture. Besides standard detection metrics, mAP@0.50 and mAP@0.50:0.95, we use TIDE error decomposition and an interpretability metric, Activation-Box Ratio, which measures how much CAM or attention mass falls inside ground-truth bounding boxes. The results show that RT-DETR-L is the strongest detector overall, mixed real-synthetic training is often competitive with real-only training, and synthetic-only training transfers to real images but typically loses accuracy. A central finding is that object focus and detection quality are not equivalent: YOLOs-small often produces high Activation-Box Ratio but weak mAP, while RT-DETR-L achieves high mAP even when its saliency becomes less object-concentrated under some synthetic regimes. This suggests that interpretability maps and detector error decomposition provide complementary information beyond aggregate mAP.

Keywords: object detection, synthetic data, synthetic-to-real transfer, DIMO, TIDE, EigenCAM, attention maps, Activation-Box Ratio

1. Introduction

Synthetic data is attractive in computer vision because it can be generated at scale and annotated automatically. However, models trained on synthetic data often face a domain gap when evaluated on real camera images. In object detection, this gap is not only a matter of lower mAP. It may also change what the model relies on: the object itself, its outline, local texture, lighting, reflections, shadows, or rendering artifacts.

The original project topic asks whether models trained on synthetic data learn the object or instead learn shortcuts such as background, texture, or shape biases. We operationalize this question for object detection on DIMO by combining standard detector evaluation with interpretability and error analysis. The report is written as a compact experimental paper: it describes the dataset, detector families, two-stage protocol, quantitative results, qualitative explanations, limitations, and reproducibility instructions.

1.2. Related Work and Background

Synthetic data is commonly used in computer vision when collecting or annotating real-world images is expensive. A central challenge in this setting is the synthetic-to-real domain gap: models trained on rendered images may perform worse on real camera images because of differences in texture, lighting, reflections, background structure, noise and object appearance. Domain randomization addresses this issue by deliberately varying rendering parameters, such as textures, lighting and camera pose, so that the real world becomes one of many possible

visual variations seen during training. This idea has been shown to support transfer from simulation to real-world visual tasks.

The DIMO dataset is well suited for this type of analysis because it contains both real-world and synthetic images of industrial metal objects. The objects are challenging because they are symmetric, textureless and reflective, while the synthetic data is generated with controlled variations that make it possible to study sim-to-real behavior more explicitly than with uncontrolled web images.

Object detection is typically evaluated using mean Average Precision, but mAP alone does not explain why a detector fails. For this reason, our analysis also uses TIDE, a toolbox designed to decompose object detection errors into categories such as classification, localization, duplicate detections, background errors and missed detections. This is aligned with our diagnostic goal: we want to understand not only which model performs best, but also what type of failure dominates under each architecture and training-data regime.

Finally, because the project focuses on what models learn from different data regimes, we include interpretability analysis through CAM-style visualizations and attention maps. Eigen-CAM is useful in this context because it visualizes principal components of learned convolutional representations and does not require gradient backpropagation through the final prediction in the same way as Grad-CAM. We combine such qualitative maps with Activation-Box Ratio to quantify whether model activation is concentrated inside the ground-truth object box or spread over background and artifacts.

2. Research Questions and Hypotheses

The study is organized around two levels of analysis: architecture behavior and data-regime behavior.

RQ1: How do different detector architecture families behave when trained and evaluated on real images from the same dataset?

RQ2: Do architecture families form separable or interpretable patterns in the space of detection quality and object focus?

RQ3: How do synthetic training regimes affect transfer to a shared real test set?

RQ4: Does higher object focus, measured by Activation-Box Ratio, imply better object detection performance?

RQ5: Which error types dominate under different architectures and data regimes?

We test the following working hypotheses:

H1: CNN-based detectors should be more sensitive to local textures, edges, shadows, and rendering artifacts, while transformer-based or hybrid detectors may rely more on broader spatial context.

H2: Real-only training should remain a strong baseline on the real test set, but mixed real-synthetic training may recover much of this performance while exposing the model to broader variability.

H3: Strong synthetic randomization can improve robustness for some models, but may also harm transfer if the synthetic distribution becomes too far from the real domain.

H4: Object focus and detection quality should be related but not identical. A model can focus on an object and still fail due to localization, classification, duplicate detections, or missed detections.

3. Dataset and Detection Task

We use DIMO as the experimental dataset. It contains real and synthetic images of industrial objects and metadata that allow controlled separation into training regimes. This is important for the project because we do not want to compare arbitrary image collections. Instead, we want to compare models trained on deliberately separated domains and synthetic variants.

The task is object detection. For each image, the detector must localize objects with bounding boxes and assign object classes. All experiments are evaluated on the same real test set. Therefore, every reported result measures performance in the target domain: real camera images.

Table 1. Training-data regimes used in Part II.

Regime	Training data	Role in the experiment
real_only	Real images only	Architecture baseline and strongest in-domain reference.
sim_all	All synthetic images	Synthetic-only transfer to the real test set.
sim_weak_dr	Weak synthetic randomization	Synthetic data with more constrained or realistic variation.
sim_strong_dr	Strong synthetic randomization	Synthetic data with stronger variation in visual conditions.
mixed	50/50 real-synthetic mix	Practical regime combining limited real data with synthetic data.

Implementation details used in the notebooks: each regime is sampled to 1,000 images. The real test set contains 100 real images not overlapping with the real-only training subset. Each training regime is split into train and validation subsets using an 80/20 split with a fixed random seed. Input image size is 224 x 224. Augmentation is not treated as an experimental variable. In the Ultralytics models, mosaic, mixup, copy-paste, color jitter, geometric transforms, and flips are disabled to keep the comparison focused on architectures and data regimes.

4. Detector Families

The project covers several detector families. Part I tests a broad architecture sweep on real-only data. Part II selects five representative models for the data-regime sweep.

Table 2. Detector families considered in the project.

Model group	Family	Role
Faster R-CNN MobileNetV3 FPN	CNN 2-stage	A lightweight two-stage CNN detector and stable baseline.
Faster R-CNN ResNet-50 FPN	CNN 2-stage	A stronger two-stage CNN detector with a ResNet backbone.
YOLOv5, YOLOv8, YOLOv9	YOLO CNN	One-stage CNN detectors with different capacity and generations.
YOLOv10, YOLO11, YOLO26	YOLO CNN+attn	YOLO-style detectors with additional attention-like or NMS-free mechanisms, depending on variant.
RT-DETR-L, RT-DETR-X	Hybrid CNN+Transformer	Hybrid detectors combining convolutional features with transformer encoder-decoder detection.
YOLOS-small	Transformer (ViT)	A ViT-style detector used as a pure transformer reference.
DETR ResNet-50	Hybrid CNN+Transformer	Classic DETR-style hybrid detector included as a reference/outlier.

5. Metrics and Interpretability

5.1 Detection quality

The main detection metrics are $\text{mAP}@0.50$ and $\text{mAP}@0.50:0.95$. $\text{mAP}@0.50$ uses a single IoU threshold of 0.50, while $\text{mAP}@0.50:0.95$ averages over stricter thresholds and is treated as the primary metric for model ranking.

5.2 TIDE error decomposition

TIDE is used to explain why detection quality drops. Instead of manually counting accuracy or recall for a detection task, TIDE decomposes detector error into interpretable categories such as classification error, localization error, classification plus localization error, duplicate detections, background false positives, and missed objects. In the report, TIDE is used as an error-analysis layer, not as a replacement for mAP.

5.3 CAM, attention, and Activation-Box Ratio

To evaluate where models focus, we compute spatial explanation maps. For CNN-based detectors we use EigenCAM or CAM-like maps. For transformer-based models, we use attention maps or attention rollout where available. All maps are resized to a common image coordinate system and compared to the ground-truth bounding boxes.

Activation-Box Ratio (ABR) is the main quantitative interpretability metric. It measures the share of activation or attention mass that falls inside ground-truth object boxes. Higher ABR means that the model explanation is more concentrated on annotated objects. However, ABR is not equivalent to detection quality. A model may focus on the correct region but still fail because of inaccurate box localization, class confusion, duplicate boxes, or missed instances.

6. Experimental Protocol

6.1 Part I: architecture baseline on real-only data

The first part isolates the effect of architecture. We train 30 detector variants on real-only data and evaluate all of them on the same real test set. This follows the instructor-aligned design: before asking what synthetic data changes, we first establish how detector architectures behave when trained on the target domain itself.

The Part I architecture sweep includes YOLO variants, Faster R-CNN variants, RT-DETR variants, DETR, and YOLOs-small. All models are evaluated with the same mAP and TIDE harness. ABR is computed on the shared real test set to relate detection quality to object focus.

6.2 Part II: data-regime transfer study

The second part isolates the effect of training-data regime. Based on Part I, five representative models are selected: Faster R-CNN MobileNetV3 FPN, YOLOv9s, YOLO26x, RT-DETR-L, and YOLOs-small. Each model is evaluated across five regimes: `real_only`, `sim_all`, `sim_weak_dr`, `sim_strong_dr`, and `mixed`. The test set remains the same real test set for all runs.

This design separates two questions that would otherwise be confounded. Part I asks what changes across architectures. Part II asks what changes when the training data changes, while keeping a much smaller set of representative architectures fixed.

7. Results Part I: Architecture Baseline on Real-Only Data

7.1 Detection quality across 30 architecture variants

Table 3. Top 10 Part I models ranked by mAP@0.50:0.95.

Model	Family	Params M	mAP@0.50	mAP@0.50:0.95	ABR	Top TIDE error
RT-DETR-L	Hybrid	31.996	0.915	0.724	0.372	Loc
RT-DETR-X	Hybrid	65.480	0.915	0.720	0.350	Loc
FRCNN-MobileNetV3	CNN 2-stage	18.956	0.879	0.574	0.392	Loc
FRCNN-ResNet50	CNN 2-stage	41.325	0.864	0.573	0.376	Loc
YOLO26x	YOLO+attn	55.640	0.788	0.505	0.269	Dupe
YOLOv9s	YOLO CNN	7.169	0.806	0.502	0.253	Loc
YOLOv8m	YOLO CNN	25.843	0.789	0.490	0.234	Loc
YOLOv9m	YOLO CNN	20.017	0.796	0.487	0.323	Loc
YOLOv5xu	YOLO CNN	97.158	0.769	0.476	0.150	Loc
YOLO26m	YOLO+attn	20.354	0.774	0.460	0.264	Dupe

Part I shows a clear performance hierarchy. RT-DETR-L and RT-DETR-X achieve the strongest results, both above 0.72 mAP@0.50:0.95. The two Faster R-CNN baselines form the next group, around 0.57 mAP@0.50:0.95. YOLO variants occupy a broad middle range, with the best variants around 0.50 mAP@0.50:0.95. YOLOs-small and DETR ResNet-50 perform substantially worse under this short fine-tuning protocol.

7.2 Architecture families, parameter count, and object focus

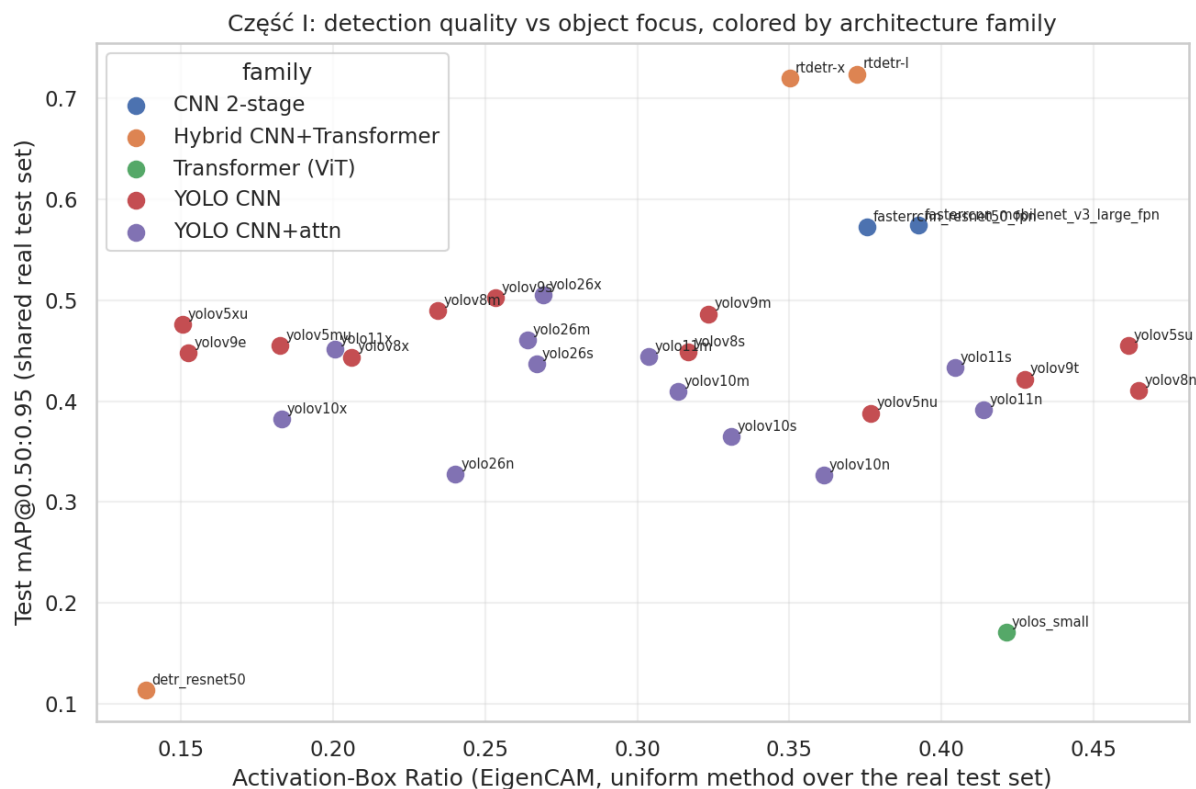


Figure 1. Part I: detection quality versus object focus. Each point is a model trained on real-only data and evaluated on the shared real test set. The plot shows that architecture families do not separate perfectly, but clear local groups and outliers are visible.

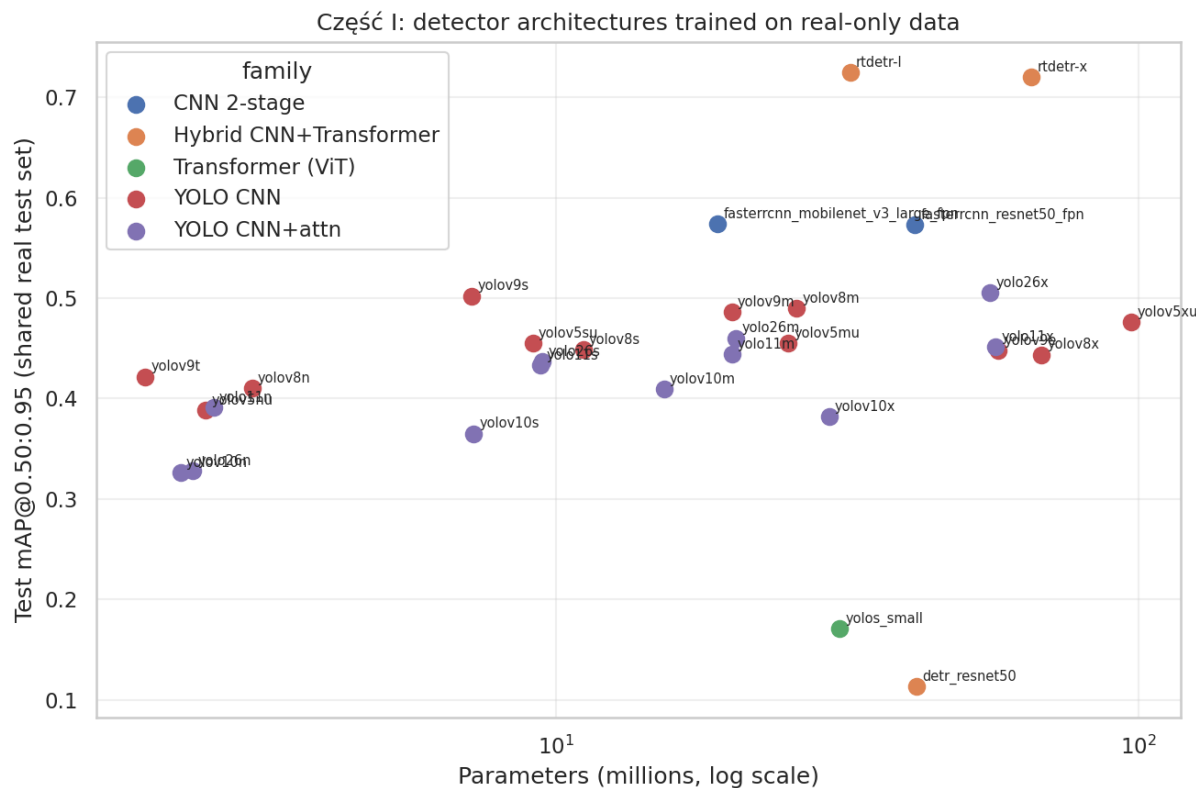


Figure 2. Part I: detection quality versus number of parameters. Larger models are not automatically better. RT-DETR variants are strong, while YOLO variants show a broad spread and DETR ResNet-50 is a low-performing outlier.

The scatter plots support a nuanced conclusion. We do not observe a clean separation where all CNNs, all transformer-based models, and all hybrids form distinct clusters. However, the plot is still informative. RT-DETR variants form a high-mAP region, Faster R-CNN variants are stable and relatively object-focused, YOLO variants form a broad cloud, and YOLOs-small is an outlier with comparatively high object focus but low detection quality.

7.3 Error decomposition with TIDE

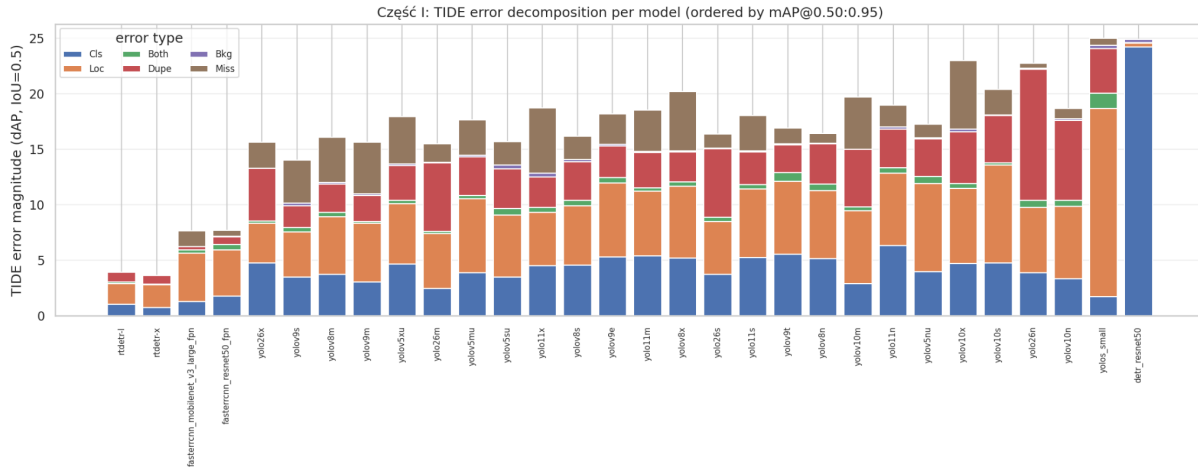


Figure 3. Part I: TIDE error decomposition per model. Most models are dominated by localization errors, while some variants show duplicate detections or missed detections as important secondary sources of error.

TIDE confirms that performance differences are not just abstract mAP gaps. Across Part I, localization is the dominant error type for most detectors. YOLO26x, one of the selected models for Part II, has duplicate detections as its dominant error in Part I. This is important because a model can have good object focus but still lose mAP through inaccurate boxes or duplicate predictions.

7.4 Interpretability examples

Full CAM-style visualization grids were generated for the real-only architecture baseline and are available in the project artifacts. In the main report, we avoid showing the full all-model grid because it is too dense for page-level interpretation. Instead, we show selected attention-map examples for Part I below, and later provide per-model qualitative case studies in Section 8.4, where predictions and CAM maps are compared across training-data regimes for selected models. These examples are illustrative; quantitative comparisons are reported separately using ABR, mAP and TIDE.

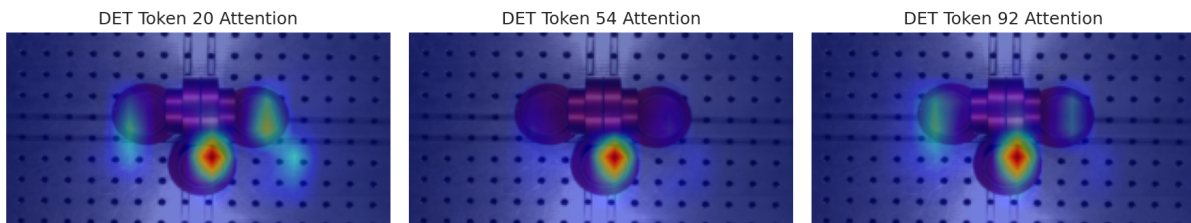


Figure 4. Part I: YOLOs attention examples. Different queries attend to different object parts or background regions, which supports the need for both qualitative and quantitative analysis.

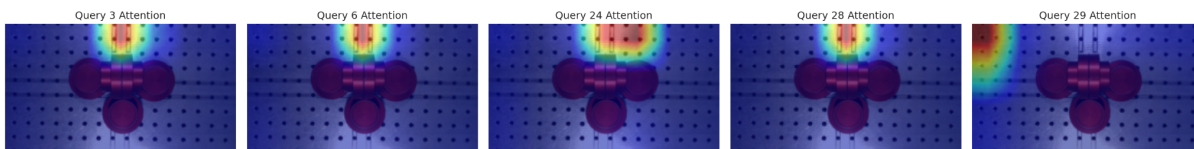


Figure 5. Part I: DETR attention examples. DETR-style attention maps show that query/token-level explanations can focus on object components, background structure, or mixed regions.

7.5 Selection of representative models for Part II

Table 4. Representative models selected for Part II.

Model	Family	Role	Params M	mAP@0.50	mAP@0.50:0.95	ABR	Top TIDE error
FRCNN-MobileNetV3	CNN 2-stage	light two-stage CNN	18.956	0.879	0.574	0.392	Loc
YOLOv9s	YOLO CNN	pure-conv one-stage CNN	7.169	0.806	0.502	0.253	Loc
YOLO26x	YOLO+attn	conv + self-attention, NMS-free	55.640	0.788	0.505	0.269	Dupe
RT-DETR-L	Hybrid	CNN backbone + transformer enc/dec	31.996	0.915	0.724	0.372	Loc
YOLOS-small	ViT	pure ViT detector	30.652	0.386	0.171	0.421	Loc

The five selected models cover complementary detector families: a lightweight two-stage CNN, a pure YOLO CNN, a YOLO variant with attention-like mechanisms, a hybrid CNN+Transformer, and a pure ViT detector. This selection allows Part II to test whether the effect of data regime is architecture-dependent.

8. Results Part II: Synthetic-to-Real Transfer Across Data Regimes

8.1 Transfer performance by training-data regime

Table 5. Part II results: five selected models across five training-data regimes.

Model	Family	Regime	mAP@0.50	mAP@0.50:0.95	ABR	Top TIDE error
FRCNN-MobileNetV3	CNN 2-stage	real_only	0.890	0.579	0.392	Loc
FRCNN-MobileNetV3	CNN 2-stage	sim_all	0.717	0.452	0.344	Loc
FRCNN-MobileNetV3	CNN 2-stage	sim_weak_dr	0.696	0.447	0.310	Loc
FRCNN-MobileNetV3	CNN 2-stage	sim_strong_dr	0.728	0.451	0.323	Bkg
FRCNN-MobileNetV3	CNN 2-stage	mixed	0.870	0.563	0.336	Loc
YOLOv9s	YOLO CNN	real_only	0.807	0.502	0.253	Loc
YOLOv9s	YOLO CNN	sim_all	0.639	0.424	0.336	Loc
YOLOv9s	YOLO CNN	sim_weak_dr	0.661	0.417	0.322	Miss
YOLOv9s	YOLO CNN	sim_strong_dr	0.675	0.445	0.197	Loc
YOLOv9s	YOLO CNN	mixed	0.773	0.509	0.220	Loc
YOLO26x	YOLO+attn	real_only	0.788	0.506	0.269	Dupe
YOLO26x	YOLO+attn	sim_all	0.664	0.434	0.195	Miss
YOLO26x	YOLO+attn	sim_weak_dr	0.655	0.423	0.297	Loc
YOLO26x	YOLO+attn	sim_strong_dr	0.697	0.464	0.230	Miss
YOLO26x	YOLO+attn	mixed	0.799	0.518	0.267	Loc
RT-DETR-L	Hybrid	real_only	0.915	0.725	0.372	Loc
RT-DETR-L	Hybrid	sim_all	0.730	0.523	0.255	Bkg
RT-DETR-L	Hybrid	sim_weak_dr	0.682	0.482	0.294	Bkg
RT-DETR-L	Hybrid	sim_strong_dr	0.729	0.531	0.274	Bkg
RT-DETR-L	Hybrid	mixed	0.895	0.680	0.224	Loc
YOLOS-small	ViT	real_only	0.464	0.157	0.600	Loc
YOLOS-small	ViT	sim_all	0.566	0.319	0.523	Loc
YOLOS-small	ViT	sim_weak_dr	0.553	0.302	0.588	Loc
YOLOS-small	ViT	sim_strong_dr	0.600	0.336	0.523	Loc
YOLOS-small	ViT	mixed	0.517	0.177	0.557	Loc

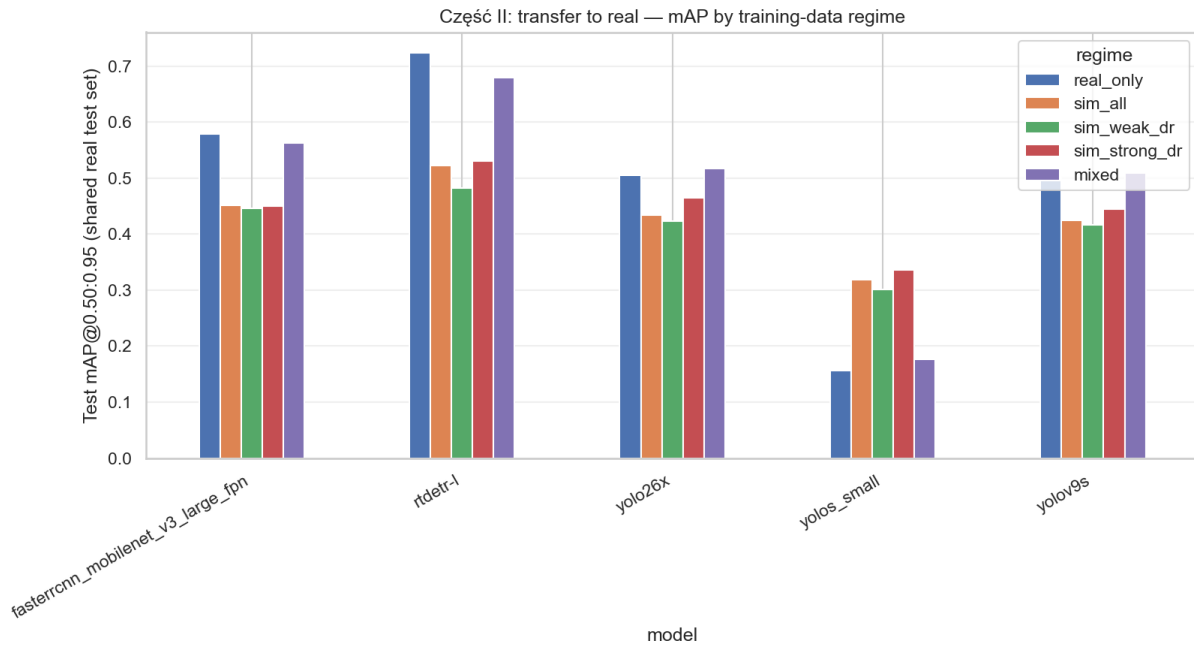


Figure 6. Part II: mAP@0.50:0.95 by model and training-data regime. Real-only is the strongest baseline for most models, mixed training is often competitive, and synthetic-only training typically loses transfer performance.

Part II confirms that data regime has a substantial effect on synthetic-to-real transfer. Real-only training remains the strongest or near-strongest regime for the best detectors. RT-DETR-L reaches 0.725 mAP@0.50:0.95 on real-only and 0.680 on mixed training. Faster R-CNN MobileNetV3 reaches 0.579 on real-only and 0.563 on mixed. For YOLO26x, mixed training slightly outperforms real-only in mAP@0.50:0.95, reaching 0.518 versus 0.506. YOLOv9s also improves slightly under mixed training, from 0.502 to 0.509. Synthetic-only regimes are useful but generally below real-only and mixed for most models, with the notable exception of YOLOS-small, which performs better on synthetic regimes than on real-only.

Table 6. Mean Part II performance by training-data regime.

Regime	Mean mAP@0.50:0.95	Mean ABR
real_only	0.494	0.378
sim_all	0.430	0.331
sim_weak_dr	0.414	0.362
sim_strong_dr	0.445	0.309
mixed	0.489	0.321

8.2 Object focus across regimes

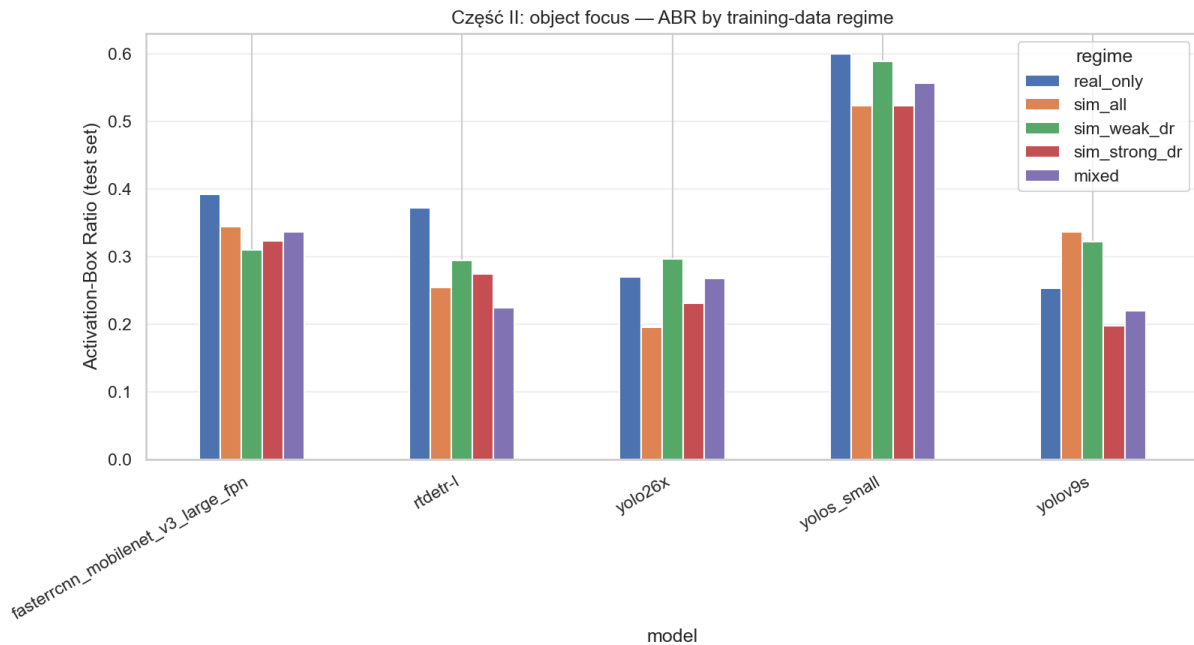


Figure 7. Part II: Activation-Box Ratio by model and training-data regime. ABR varies substantially across models and regimes, but it does not follow the same pattern as mAP.

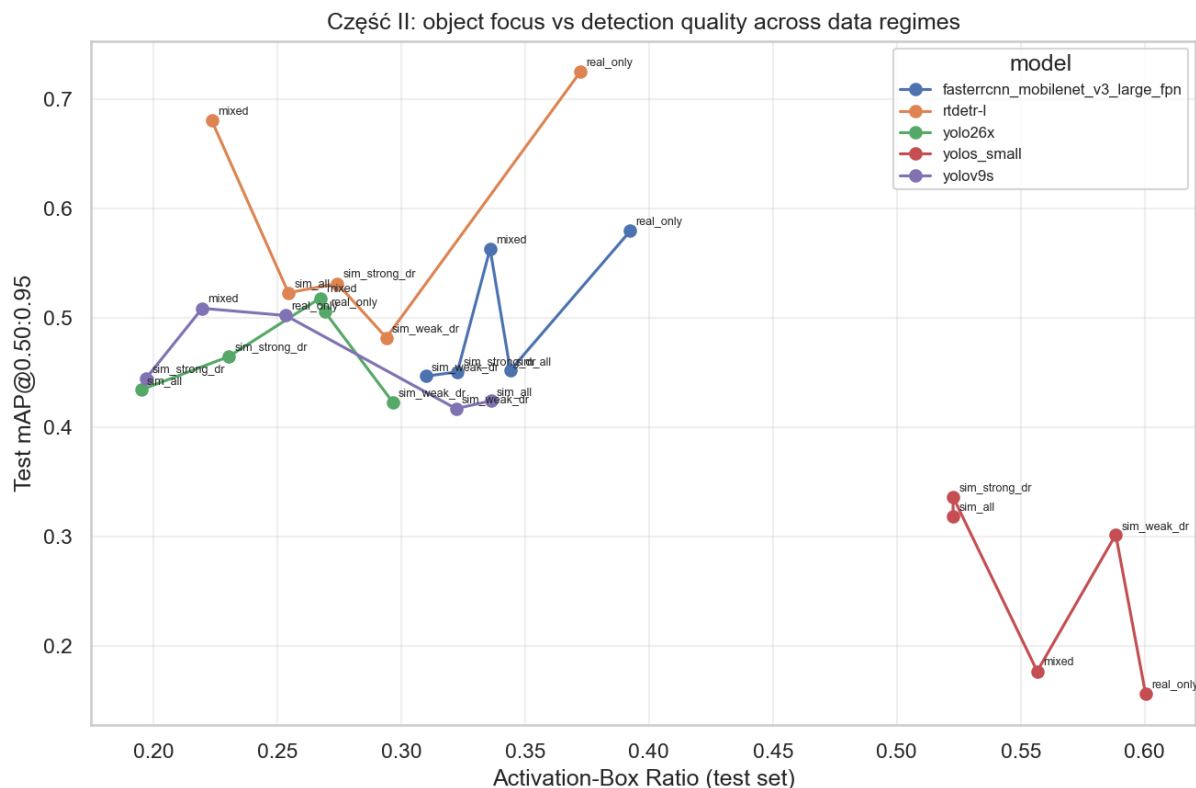


Figure 8. Part II: object focus versus detection quality across regimes. The trajectories show that higher ABR does not necessarily imply higher detection quality.

The strongest non-trivial finding of Part II is that ABR and mAP are not equivalent. YOLOS-small has the highest mean ABR across regimes, approximately 0.56, but the lowest mean mAP@0.50:0.95, approximately 0.26. In contrast, RT-DETR-L has the strongest detection performance, approximately 0.59 mean mAP@0.50:0.95 across regimes, but a much lower mean ABR, approximately 0.28. This shows that a model can concentrate its saliency

inside object boxes and still be a weak detector. Object focus is therefore a useful diagnostic signal, but not a substitute for detection metrics.

8.3 Error decomposition across data regimes

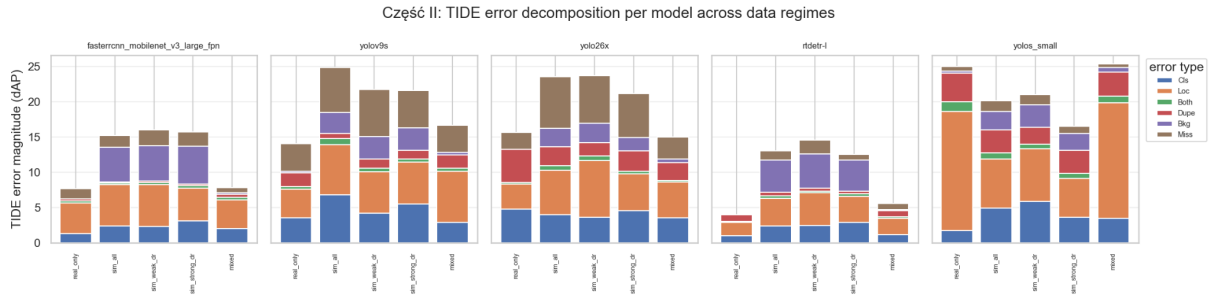


Figure 9. Part II: TIDE error decomposition across models and data regimes. The dominant error types differ by architecture and regime. Localization remains important, while background, missed detections, false positives, and duplicates explain many of the synthetic-transfer failures.

The TIDE plots show that synthetic-to-real transfer failures are model-specific. RT-DETR-L is consistently strong, but synthetic regimes increase background-related errors in the table-level summary. YOLO26x and YOLOv9s show sensitivity to missed detections or localization depending on the regime. YOLOs-small remains dominated by localization-related problems despite high object focus. Faster R-CNN MobileNetV3 is comparatively stable, though its synthetic regimes introduce more background or localization error than real-only and mixed training.

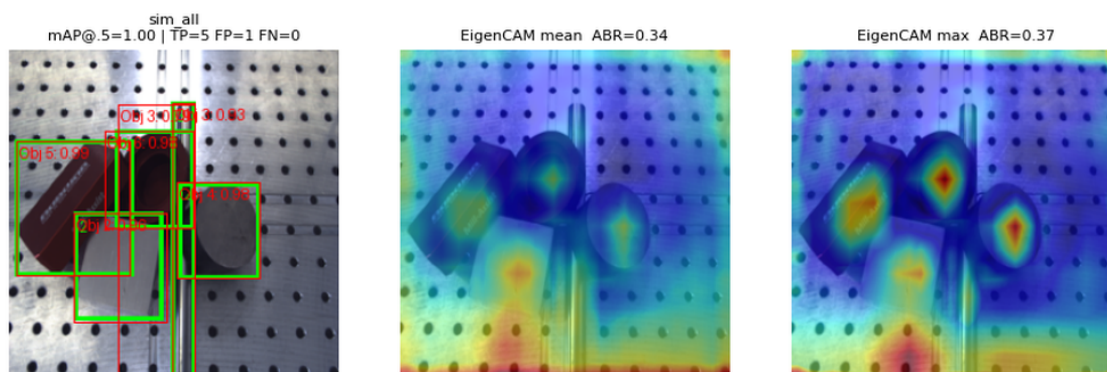
8.4 Qualitative examples: failure cases, predictions and CAMs across regimes

The following qualitative examples connect the quantitative results to visible detector behavior. First, we show representative failure cases selected from the prediction visualizations to make the TIDE error decomposition more concrete. Green boxes denote ground-truth annotations and red boxes denote model predictions. We then compare predictions and CAM or attention maps across training-data regimes for selected models on the same real test image.

Qualitative Error Examples

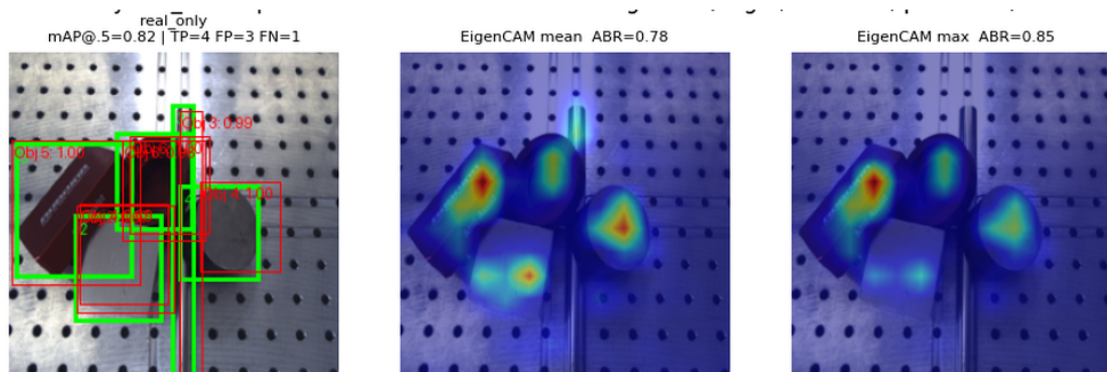
A. RT-DETR-L, `sim_all`: background false positive

One extra prediction appears on a vertical rail-like background structure, consistent with the TIDE background-error signal for synthetic-only RT-DETR.



B. YOLO5-small, `real_only`: high ABR but incorrect detections

Despite object-centric CAMs, the detector produces several false positives and one missed object. High ABR alone does not guarantee correct detection.



C. YOLO26x, `sim_all`: missed detection and dispersed activation

The model misses one object, while the CAM contains hot regions on the background grid and vertical structures, matching the missed-detection pattern.

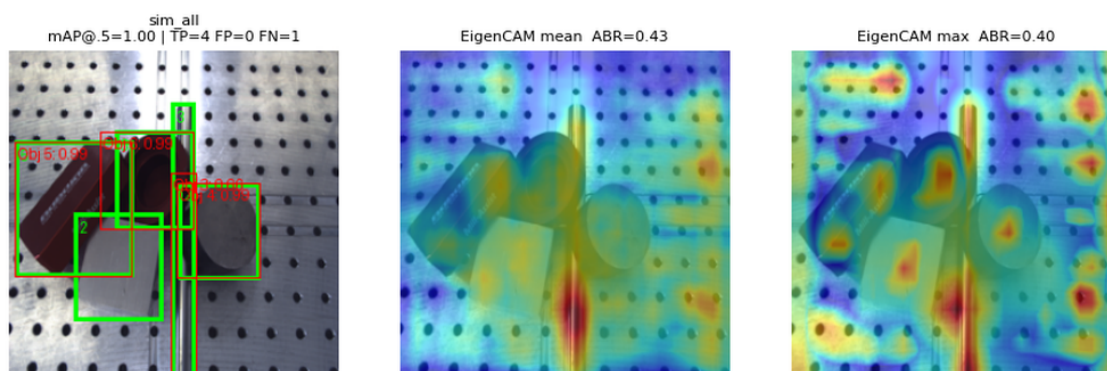


Figure 10. Representative qualitative failure cases.

- (A) RT-DETR-L trained on `sim_all` produces one additional prediction on a vertical rail-like background structure, while all ground-truth objects are detected. This is consistent with the increased background-error signal observed for synthetic-only RT-DETR variants in TIDE.
- (B) YOLO5-small trained on `real_only` produces several false positives and one missed object despite high ABR values. This illustrates that object-centric activation does not necessarily translate into correct detection.
- (C) YOLO26x trained on `sim_all` misses one object, while its CAM contains dispersed hot regions on the background grid and vertical structures. This matches the missed-detection pattern observed for this model and regime.

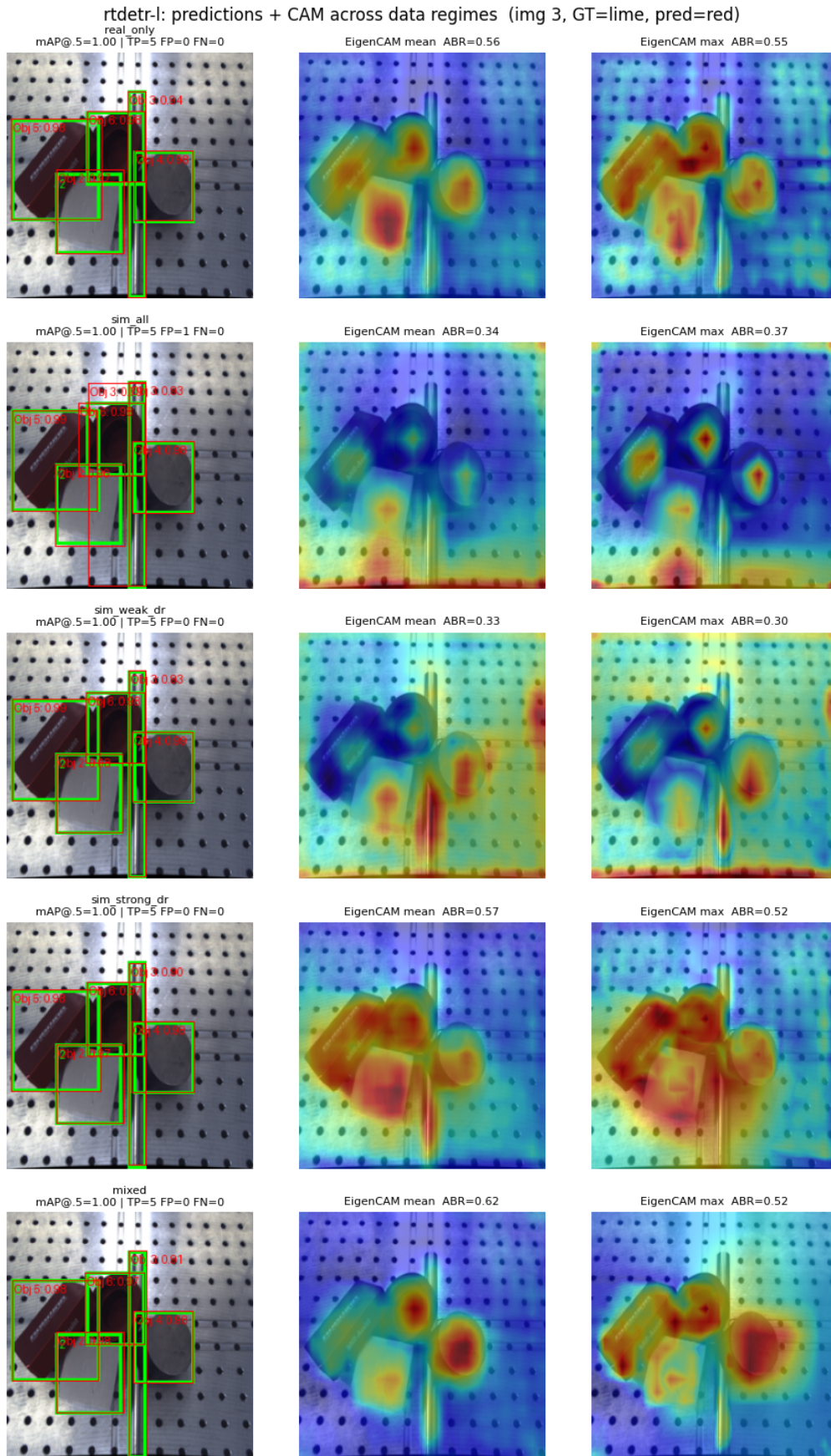


Figure 11. RT-DETR-L across regimes. The model keeps strong detection quality across regimes, but its CAM distribution changes substantially. This supports the claim that training regime affects explanation maps even when detection quality remains high.

fasterrcnn_mobilenet_v3_large_fpn: predictions + CAM across data regimes (img 3, GT=lime, pred=red)

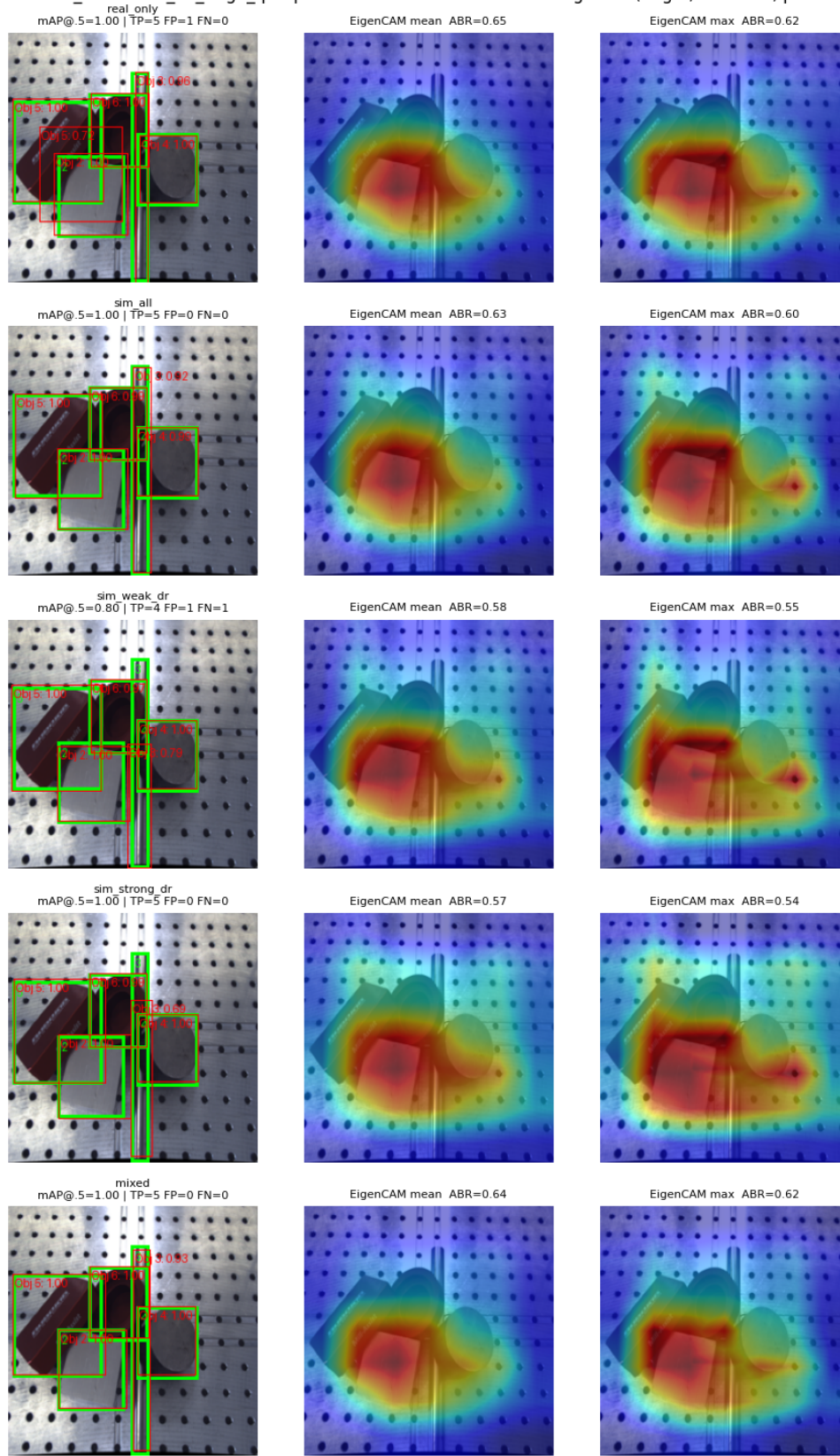


Figure 12. Faster R-CNN MobileNetV3 across regimes. The model is relatively stable and its CAMs remain concentrated around object regions, making it a useful CNN baseline.

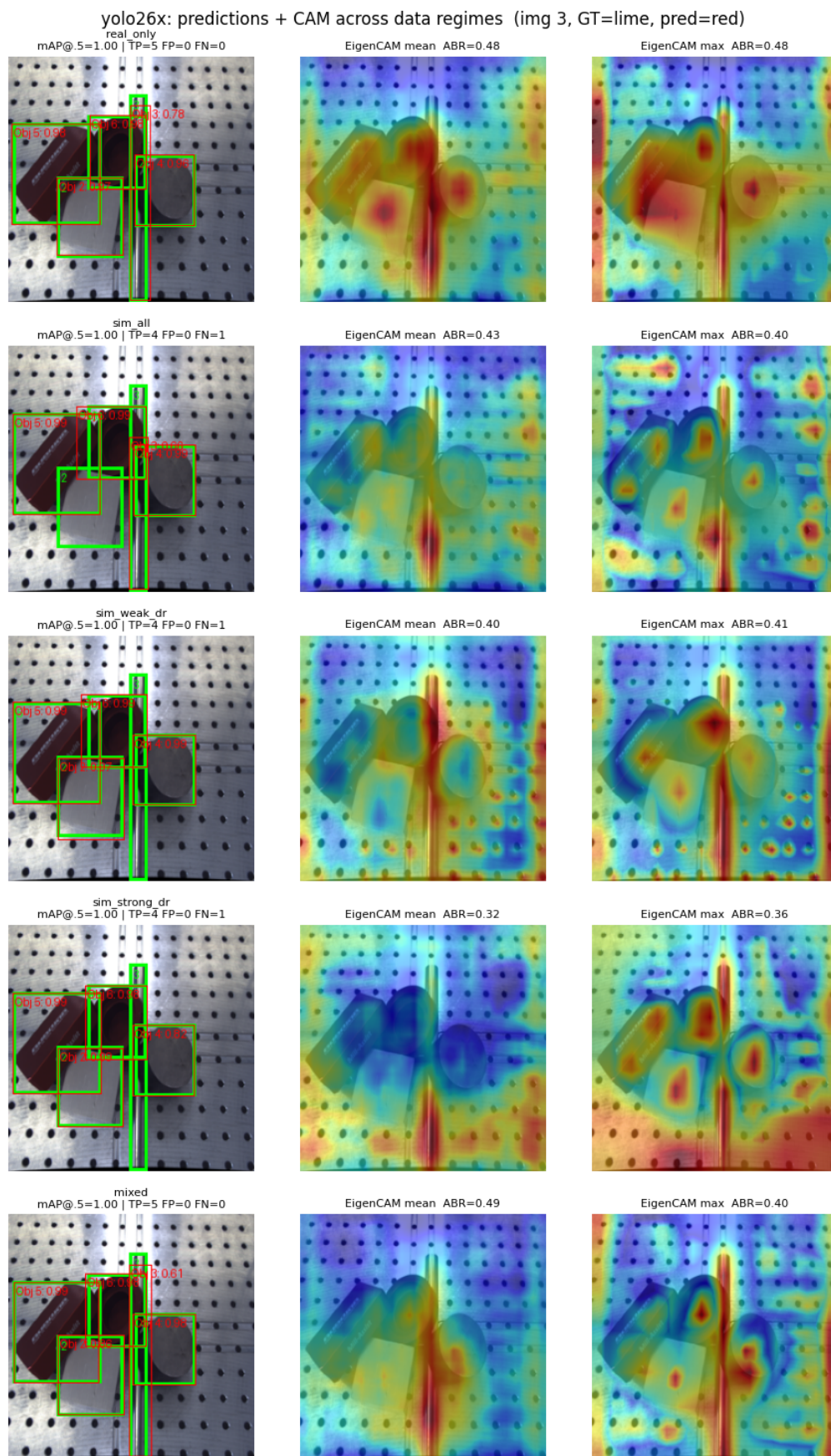


Figure 13. YOLO26x across regimes. Mixed and real-only regimes produce stronger behavior, while synthetic regimes introduce missed detections and saliency shifts toward background structures or partial objects.

yolos_small: predictions + CAM across data regimes (img 3, GT=lime, pred=red)

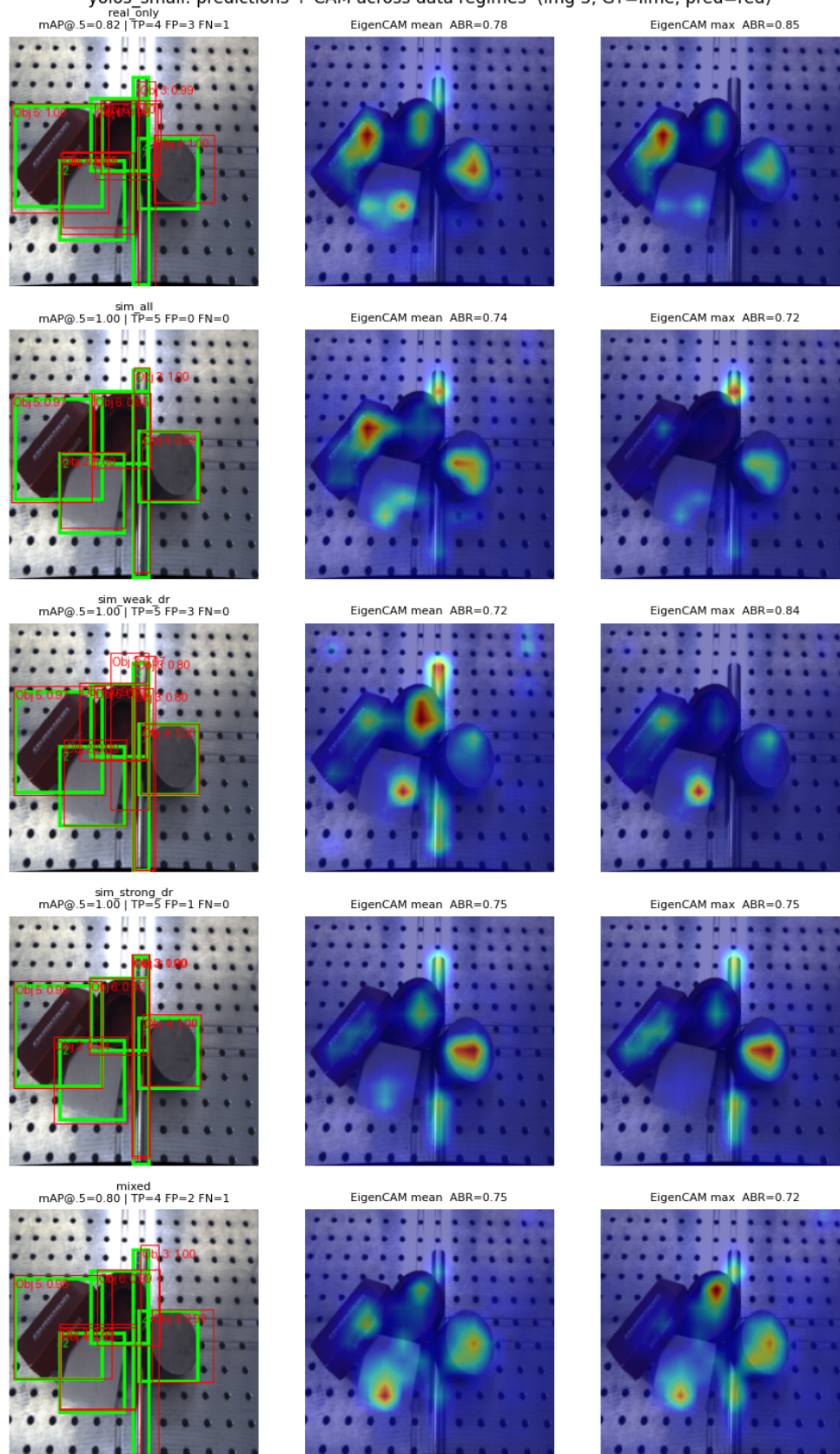


Figure 14. YOLO-small across regimes. The model often shows high object-centered saliency, but still produces weaker detection performance, illustrating the gap between object focus and detector quality.

9. Discussion

9.1 Architecture matters before data regime matters

The two-stage experimental design was useful. If all models and all data regimes were analyzed at once, it would be difficult to determine whether a result came from architecture or data. Part I provides an architecture baseline: RT-DETR variants are the strongest, Faster R-CNN is stable, YOLO variants vary widely, and YOLOs-small is a low-mAP but high-focus outlier. Part II then shows how selected representatives react to different data regimes.

9.2 Mixed real-synthetic training is a strong practical regime

Mixed training is one of the most practically relevant results. It is competitive with real-only training for several models and outperforms synthetic-only regimes in most cases. For YOLO26x and YOLOv9s, mixed training slightly improves mAP@0.50:0.95 relative to real-only. For RT-DETR-L and Faster R-CNN MobileNetV3, mixed training remains close to real-only and substantially above the synthetic-only regimes. This suggests that synthetic data is most useful when it complements real data rather than fully replacing it.

9.3 Stronger synthetic randomization is not universally better

Strong synthetic randomization sometimes helps and sometimes harms. For RT-DETR-L and YOLOs-small, `sim_strong_dr` performs better than `sim_weak_dr` in mAP@0.50:0.95. For Faster R-CNN MobileNetV3 and YOLOv9s, differences between synthetic regimes are smaller or inconsistent. The conclusion is not that strong randomization is always better. The better conclusion is that randomization changes both transfer performance and model focus in architecture-dependent ways.

9.4 Object focus is not enough

A key finding is that ABR should not be interpreted as a standalone quality metric. YOLOs-small is the clearest example: it has high ABR but weak mAP. This means that a detector can attend to an object region but still fail to produce accurate boxes, correct classes, or non-duplicated predictions. The most useful interpretation comes from combining mAP, ABR, TIDE, and qualitative examples.

9.5 Shortcut learning and synthetic data

The qualitative CAMs suggest several possible shortcuts: sensitivity to background grid patterns, vertical rails, shadows, reflections, and partial object components. However, these should be treated as diagnostic hypotheses rather than definitive causal proof. The value of the analysis is that it identifies where errors and attention shifts occur, and connects them to changes in training data regime.

10. Limitations

The training subsets are intentionally small to allow many experiments. This supports trend analysis but should not be interpreted as exhaustive benchmarking.

The real test set contains 100 images. This is sufficient for controlled comparison in this project, but larger test sets would give more stable estimates.

CAM and attention maps are not mathematically identical across architectures. We use them as comparable spatial explanation maps after normalization, but the comparison remains approximate.

ABR measures spatial overlap between activation and ground-truth boxes, but it does not measure whether the model used the correct semantic feature inside the box.

The project does not claim that one architecture family is universally better. The conclusions are specific to DIMO, the selected training protocol, and short fine-tuning.

Embedding-space analyses such as UMAP/t-SNE were not the central analysis in the final protocol. The revised protocol prioritized architecture scatter plots, ABR, TIDE, CAMs, attention maps, and qualitative error examples.

11. Reproducibility Instructions

The project is reproducible through the notebooks provided in the GitHub repository and the external Google Drive artifact folder. The GitHub repository contains the code notebooks used for data preparation, training, evaluation and visualization. Due to file size constraints, large DIMO subsets, trained model checkpoints and full experiment artifacts are stored in the linked Google Drive folder.

The repository and artifact folder should include:

- notebooks for DIMO loading, subset construction and training-data regime preparation;
- notebooks for Part I architecture baseline experiments;
- notebooks for Part II training-regime experiments;
- evaluation and visualization code for mAP, TIDE, ABR, CAM-style maps, attention maps and plots;
- lightweight result figures and tables used in this report;
- CSV files with experiment summaries, where available;
- a README file describing the repository structure, external artifacts and execution order.

Reproduction outline:

1. Clone the GitHub repository.
2. Open the provided notebooks in Google Colab or another Jupyter-compatible environment.
3. Restore the prepared DIMO subset and model checkpoints from the linked Google Drive artifact folder, or run the dataset preparation cells if rebuilding the subsets from scratch.
4. Run the Part I notebook to reproduce the real-only architecture baseline, or load the provided fine-tuned checkpoints.
5. Run the Part II notebook to evaluate the selected models across training-data regimes: `real_only`, `sim_all`, `sim_weak_dr`, `sim_strong_dr` and `mixed`.
6. Run the evaluation and visualization cells to regenerate mAP tables, TIDE error decompositions, ABR metrics, scatter plots and qualitative CAM/attention figures.

12. Conclusions

This project studied what object detectors learn from synthetic data by combining architecture comparison, synthetic-to-real transfer, error decomposition, and interpretability. The revised two-part protocol made the analysis cleaner: Part I established how detector architectures behave on real data, and Part II studied how selected architectures respond to different training-data regimes.

The main empirical conclusions are: RT-DETR-L is the strongest detector overall; Faster R-CNN MobileNetV3 is a stable and interpretable CNN baseline; YOLO variants show broad variability; mixed real-synthetic training is often competitive with real-only training; synthetic-only training transfers to real images but usually loses accuracy; and object focus measured by ABR does not directly imply high detection quality. The final conclusion is that synthetic data can help object detection, especially in mixed regimes, but understanding its effect requires more than mAP. TIDE and spatial explanation maps reveal whether failures arise from localization, duplicates, background, missed detections, or attention shifts toward non-object regions.

References

- Ren, S., He, K., Girshick, R., and Sun, J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS, 2015.
- Carion, N. et al. End-to-End Object Detection with Transformers. ECCV, 2020.
- Fang, Y. et al. You Only Look at One Sequence: Rethinking Transformer in Vision through Object Detection. NeurIPS, 2021.
- Selvaraju, R. R. et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. ICCV, 2017.
- Bolya, D. et al. TIDE: A General Toolbox for Identifying Object Detection Errors. ECCV, 2020.
- Ultralytics. Models Supported by Ultralytics. Documentation for YOLO and RT-DETR model variants. Available at: <https://docs.ultralytics.com/models/>
- Ultralytics. RT-DETR: Real-Time Detection Transformer. Available at: <https://docs.ultralytics.com/models/rtdetr/>
- De Roovere, P., Moonen, S., Michiels, N., & Wyffels, F. Dataset of Industrial Metal Objects. Project website. Available at: <https://pderoovere.github.io/dimo/>
- De Roovere, P., Moonen, S., Michiels, N., & Wyffels, F. Dataset of Industrial Metal Objects. arXiv:2208.04052. Available at: <https://arxiv.org/abs/2208.04052>
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. arXiv:1703.06907, 2017.
- Muhammad, M. B., & Yeasin, M. Eigen-CAM: Visual Explanations for Deep Convolutional Neural Networks. SN Computer Science, 2021.